

Bike Rental and Ride-Sharing Platform

Final Report

Module: SE1020 Object Oriented Programming

Assignment: Project

Technology Stack: Java, Spring Boot, Thymeleaf, Bootstrap, File Handling

Repository: <https://github.com/ashifahmed-924/OOP-bike-rental-system>

Student Name: _____

Student ID: _____

Date: 2026-04-10

1. Introduction

This project is a web-based Bike Rental and Ride-Sharing Platform developed for the SE1020 Object Oriented Programming module. The system allows users to register, log in, manage bikes, create ride bookings, and handle ride-sharing requests. The backend is implemented using Java and Spring Boot, while file handling is used for data persistence instead of a database.

2. Objectives

- Apply object-oriented programming concepts such as encapsulation, inheritance, abstraction, and polymorphism.
- Implement CRUD operations for user management, bike management, and ride bookings.
- Develop a user-friendly interface for different system roles.
- Use file read/write operations for persistent storage.
- Track development using Git and GitHub.

3. Technologies Used

Backend

- Java
- Spring Boot
- Spring MVC
- Thymeleaf

Frontend and Tooling

- HTML / CSS
- Bootstrap
- Maven
- GitHub

4. Main Features

- User registration and login with role-based access.
- Admin user management with create, search, update, and delete operations.
- Operator bike management with add, update, and delete operations.
- Rider bike booking and ride-sharing request handling.
- Operator ride booking overview limited to bikes added by that operator.
- File-based persistence using text files.

5. Project Structure

- `src/main/java/app` - Spring Boot application and controllers

- `src/main/java/model` - domain models and polymorphic user types
- `src/main/java/dao` - file-based data access objects
- `src/main/resources/templates` - Thymeleaf HTML templates
- `src/main/resources/static/assets` - CSS and image assets
- `src/main/webapp/data` - text files for persistence

6. Object-Oriented Programming Concepts

Encapsulation

Encapsulation is used in the model classes through controlled access to data fields using getters and setters. Examples include `Bike`, `RideBooking`, and `ShareRideRequest`.

Inheritance

Inheritance is used through the class hierarchy `Person` -> `User` -> `AdminUser` / `OperatorUser` / `RiderUser`. This structure promotes reuse of common properties such as username and password.

Polymorphism

Polymorphism is implemented using specialized user subclasses. The system works with the abstract `User` type, while each subclass provides its own implementation of `getRole()` and `getDashboardPath()`. This allows the application to redirect users based on behavior defined by the actual runtime object.

Abstraction

The classes `Person` and `User` are abstract and define the common structure and required methods for all user types.

7. CRUD Operations

Module	Create	Read	Update	Delete
Users	Register / Add User	Search / List Users	Update User	Delete User
Bikes	Add Bike	List Bikes	Update Bike	Delete Bike
Ride Bookings	Create Booking	List Bookings	Update Booking	Delete Booking

8. File Handling

The application stores data using text files instead of a relational database.

- `users.txt` - stores usernames, passwords, and roles
- `bikes.txt` - stores bike records
- `ride-bookings.txt` - stores ride booking and share request details

File operations are handled through UserDAO, BikeDAO, and RideBookingDAO.

9. Class Diagram

The following Mermaid code represents the core class design of the system.

```
classDiagram
    class Person {
        <<abstract>>
        #String username
        #String password
        +getRole() String
        +getDashboardPath() String
    }

    class User {
        <<abstract>>
        +toString() String
    }

    class AdminUser
    class OperatorUser
    class RiderUser
    class UserFactory
    class Bike
    class RideBooking
    class ShareRideRequest
    class UserDAO
    class BikeDAO
    class RideBookingDAO
    class BaseWebController
    class PageController
    class AuthController
    class UserController
    class BikeController
    class RideBookingController

    Person <|-- User
    User <|-- AdminUser
    User <|-- OperatorUser
    User <|-- RiderUser
    UserFactory ..> User
    UserDAO ..> UserFactory
    RideBooking o-- ShareRideRequest
    BaseWebController <|-- PageController
    BaseWebController <|-- AuthController
    BaseWebController <|-- UserController
    BaseWebController <|-- BikeController
    BaseWebController <|-- RideBookingController
```

10. User Interfaces

- Home page
- Login and registration pages
- Admin user management page
- Operator bike management page
- Ride booking management page

Screenshots can be inserted in this section before submission if required by the lecturer.

11. Challenges Faced

- Migrating the project from JSP/Servlet architecture to Spring Boot.
- Converting JSP pages into Thymeleaf templates.
- Preserving file-based data storage after the framework migration.
- Adding a clearer polymorphism implementation for viva evaluation.

12. Conclusion

The project successfully demonstrates object-oriented programming concepts, file handling, role-based web development, and multiple CRUD operations using Java and Spring Boot. The final system provides a complete bike rental and ride-sharing workflow while remaining aligned with the assignment requirements.

13. Git Repository and Commit History

Repository URL: <https://github.com/ashifahmed-924/OOP-bike-rental-system>

Version control for the project was managed using Git and GitHub. The commit history shows the progress of the application from the initial implementation to the final Spring Boot and Thymeleaf migration.

Commit ID	Message
681ffa4	Convert app to Spring Boot and Thymeleaf
10d8d84	first commit

Git Log

```
681ffa4 Convert app to Spring Boot and Thymeleaf
10d8d84 first commit
```